PRACTICA 1

Sistemas Operativos

GUILLERMO GIMENO MARTINEZ
https://github.com/guillemrtz06/SO

INDICE

1.1. Enunciado del Ejercicio 5
1.1.1. Estructura del Árbol de Procesos 5
1.2. Identificación y Justificación de Tareas 5
1.2.1. Recepción y Validación de Argumentos 5
1.2.2. Generación de la Estructura Horizontal (Columnas) 5
1.2.3. Generación de la Estructura Vertical (Filas) 6
1.2.4. Espera Controlada y Mantenimiento del Árbol 6
1.3. Análisis Detallado del Código Fuente 6
1.3.1. Fase de Inicialización y Validación 6
1.3.2. Creación de las Columnas (Bucle Horizontal) 6
1.3.3. Control del Padre Principal (Proceso raíz) 7
1.3.4. Creación de las Filas (Bucle Vertical) 7
1.3.5. Temporización Final (Procesos Hoja) 8
1.4. Diagrama de secuencia 9
Ejercicio 1 (Apartado b) 10
1.5. Enunciado del Ejercicio 10
1.5.1. Estructura del Árbol de Procesos 10
1.6. Identificación y Justificación de Tareas 10
1.6.1. Recepción y Validación de Argumentos 10
1.6.2. Generación de la jerarquía de procesos anidada: 10
1.6.3. Temporización 10
1.6.4. Ejecución del comando pstree 11
1.6.5. Destrucción controlada 11
1.7. Análisis Detallado del Código Fuente 11
1.7.1. Funciones manejadoras de señales 11
1.7.2. Función parse_args() 12
1.7.3. Funciones de los procesos hoja 12
1.7.4. Funciones de gestión del proceso B 13
1.7.5. Función crear_proceso_a() 14
1.7.6. Función main() 14
1.8. Diagrama de secuencia 15

Ejercicio 2 16
2.1. Enunciado del Ejercicio 16
2.2. Identificación y Justificación de Tareas 16
2.2.1. Recepción y Validación de Argumentos 16
2.2.2. Cálculo de fragmentos 16
2.2.3. Mecanismo de división y comunicación (Padre e Hijos) 16
2.3. Análisis Detallado del Código Fuente 17
2.3.1. Función parse_args() 17
2.3.2. Función inicializar_datos_archivo() 17
2.3.3. Función abrir_archivo_origen() 18
2.3.4. Función crear_y_escribir_fragmento() 18
2.3.5. Función leer_y_enviar_datos() 19
2.3.6. Función generar_hijos_y_fragmentar() 19
2.3.7. Función main() 20
2.4. Diagrama de secuencia 21
Ejercicio 3 22
3.1. Enunciado del Ejercicio 22
3.4.1. Estructura del Árbol de Procesos 22
3.2. Identificación y Justificación de Tareas 22
3.2.1. Recepción y Validación de Argumentos 22
3.2.2. Gestión de la Memoria Compartida 22
3.2.3. Creación de la rama vertical (Padres): 22
3.2.3. Creación de las hojas (Subhijos) 23
3.3. Análisis Detallado del Código Fuente 23
3.3.1. Funciones de configuración y memoria 23
3.3.2. Funciones auxiliares 24
3.3.3. Funciones de creación de procesos (Motor principal) 25
3.4. Diagrama de secuencia 27

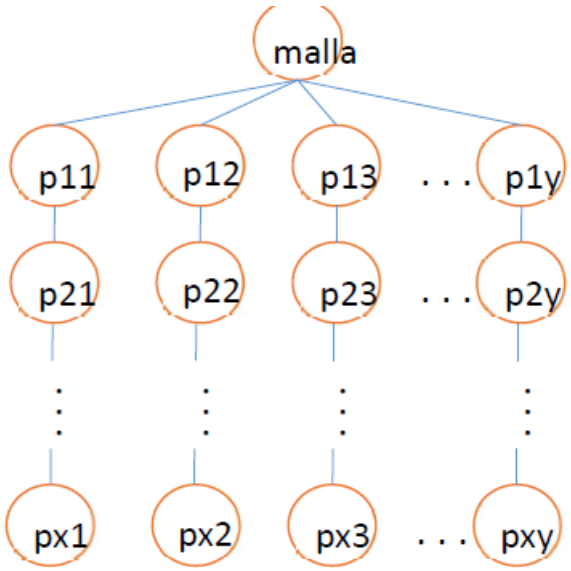
Ejercicio 1 (Apartado a)

1.1. Enunciado del Ejercicio

Realiza un programa llamado **malla.c** que produzca un árbol de procesos específico. El programa recibirá dos argumentos, ‘x’ e ‘y’, que representan el número de filas y columnas respectivamente. Para comprobar la estructura de procesos que realmente estamos creando, el enunciado indica que se debe emplear la llamada al sistema "pstree –c"

1.1.1. Estructura del Árbol de Procesos

La jerarquía de procesos a realizar es la siguiente:



1.2. Identificación y Justificación de Tareas

La arquitectura del programa se deriva de un análisis de los requisitos del enunciado. Cada tarea se traduce en una decisión de diseño para poder formar la malla bidimensional.

1.2.1. Recepción y Validación de Argumentos

Primero nos asegurarnos de que el usuario le pasa los datos bien al programa. Comprobamos que introducimos exactamente tres argumentos en total (el nombre del programa más los dos números). Si todo va bien, los pasamos de texto a entero usando atoi().

1.2.2. Generación de la Estructura Horizontal (Columnas)

Para la primera parte de la malla, el proceso principal tiene que encargarse de crear tantos hijos directos como columnas le hayamos pedido. Para ello utilizamos un bucle y llamadas a fork(). Los hijos rompen el bucle con un break en cuanto nacen.

1.2.3. Generación de la Estructura Vertical (Filas)

Una vez que tenemos esa primera línea horizontal, cada uno de esos procesos tiene que formar su propia columna. Para conseguir esa cadena vertical, cada proceso crea un único hijo, espera a que termine y luego se cierra.

1.2.4. Espera Controlada y Mantenimiento del Árbol

Para poder visualizar el árbol de procesos, estos no pueden terminar su ejecución de forma instantánea. Para ello utilizamos alarm() combinada con pause() en los procesos hoja para mantener la estructura viva el tiempo suficiente para ejecutar el comando pstree -c. Además, cada padre debe usar wait(NULL) para esperar a sus hijos y evitar la creación de procesos "zombie" y la destrucción prematura del árbol.

1.3. Análisis Detallado del Código Fuente

A continuación, se desglosa el funcionamiento lógico del programa malla.c y las llamadas al sistema utilizadas en cada bloque

1.3.1. Fase de Inicialización y Validación

El programa comienza evaluando los argumentos pasados por la consola.

- Llamadas al Sistema Utilizadas: **printf(), exit(), atoi()**

```c
//comprobamos que los argumentos son correctos
if (argc != 3) {
    printf("Uso: ./malla <filas_x> <columnas_y>\n");
    exit(1);
}

//convertios los argumentos 'x' e 'y en numeros enteros
int x = atoi(argv[1]);
int y = atoi(argv[2]);
```

Se asegura de que existan los dos parámetros x e y, y se convierten a enteros para controlar los bucles posteriores.

1.3.2. Creación de las Columnas (Bucle Horizontal)

El proceso principal (raíz) se encarga de crear las cabeceras de cada columna.

- Llamadas al Sistema Utilizadas: **fork(), perror(), exit()**

```c
// Creamos 'y' columnas
for(int i = 0; i < y; i++){
    pid = fork();

    if(pid < 0){
        perror("Error en el fork");
        exit(1);
    }

    if(pid == 0){
        break;
    }
}
```

El proceso malla original itera y veces. Cuando fork() devuelve 0, significa que nos encontramos en el proceso hijo recién creado. Mediante la instrucción break, garantizamos que el hijo escape de este bucle horizontal y no siga creando más procesos paralelos.

1.3.3. Control del Padre Principal (Proceso raíz)

El proceso original debe esperar a que todas las columnas que ha creado terminen antes de finalizar.

- Llamadas al Sistema Utilizadas: **wait(), exit().**

```c
if (pid > 0) {

    for (int j = 0; j < y; j++) {
        wait(NULL);
    }
    exit(0);
}
```

Al evaluar pid > 0, nos aseguramos de que solo el creador de la primera fila entra en esta condición. Imprime su PID para facilitar la orden pstree -c requerida por el enunciado, y entra en un bucle wait(NULL) para recolectar el estado de cada uno de sus y hijos directos.

1.3.4. Creación de las Filas (Bucle Vertical)

Los procesos que escaparon del primer bucle (las cabeceras de columna) proceden a crear la estructura hacia abajo.

- Llamadas al Sistema Utilizadas: **fork(), perror(), wait(), exit().**

```c
// El proceso hijo crea 'x' filas
for(int i = 1 ; i < x; i++){
    pid = fork();

    if(pid < 0){
        perror("Error en el fork vertical");
        exit(1);
    }

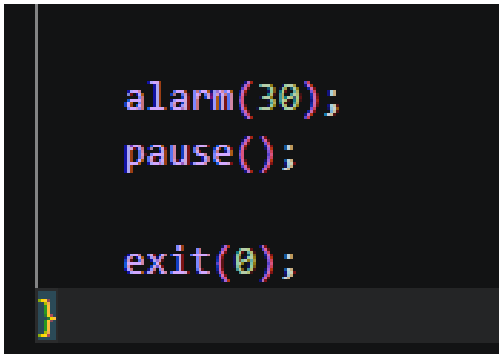
    if(pid > 0){ // El proceso hijo espera a que su hijo termine antes de salir
        wait(NULL);
        exit(0);
    }
}
```

Este bucle itera x-1 veces. En cada vuelta, el proceso crea un hijo. Si el código está ejecutándose en el proceso padre de ese nivel (pid > 0), esta llama a wait(NULL) y, una vez que su hijo termina, ejecuta exit(0). De este modo, la ejecución continúa en el nuevo proceso hijo, quien dará la siguiente vuelta al bucle, creando así una cadena descendente.

1.3.5. Temporización Final (Procesos Hoja)

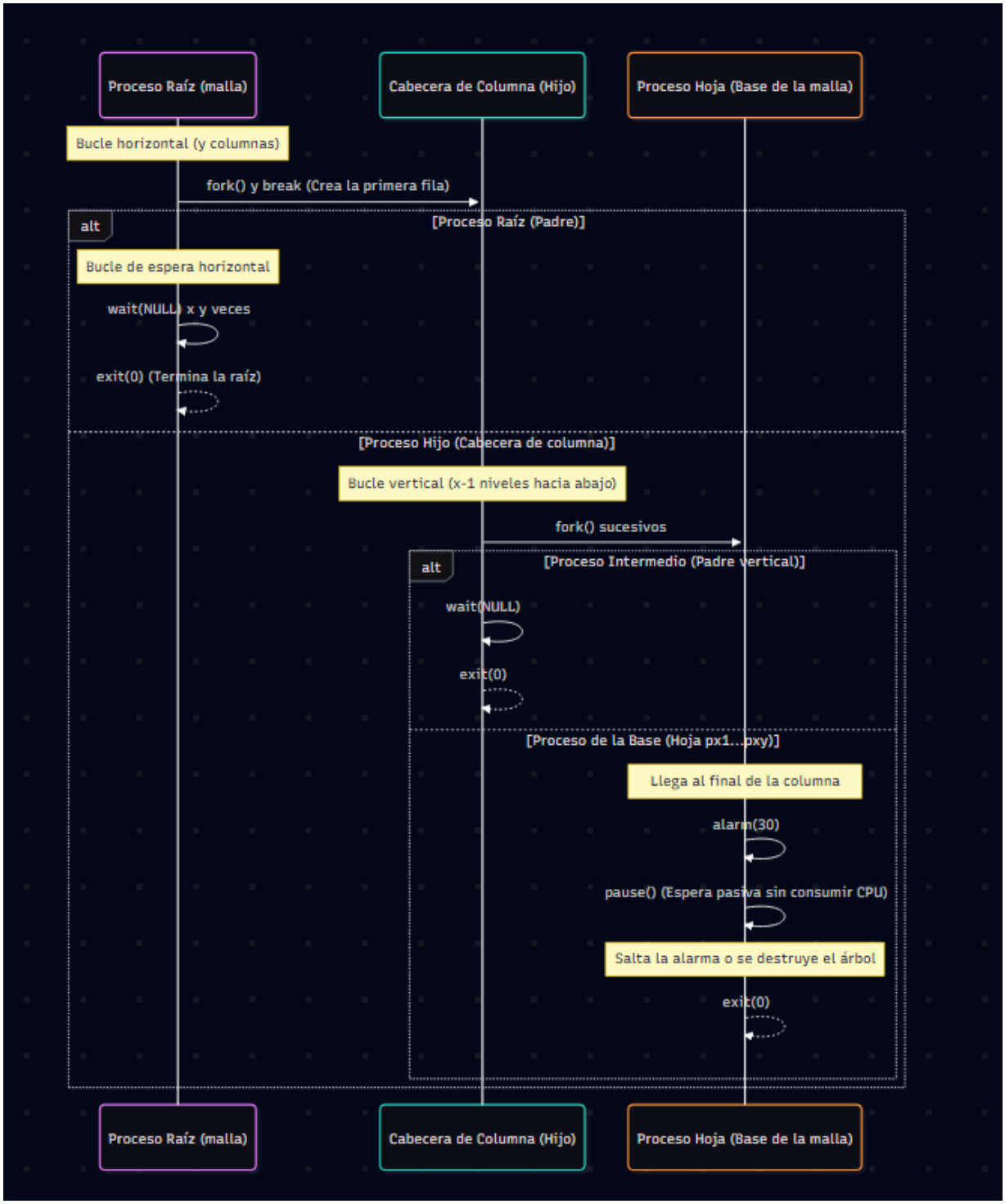
Una vez que el último hijo de cada columna ha sido creado (la fila x), escapa del bucle vertical.

- Llamadas al Sistema Utilizadas: **alarm(), pause(), exit().**



Los procesos situados en la base de la malla ($px1, px2, ..., pxy$) alcanzan estas instrucciones finales. Mediante alarm(30) programan el envío de una señal automática para dentro de 30 segundos, y con pause() suspenden su ejecución de forma pasiva y eficiente sin consumir ciclos de CPU. Al bloquearse de este modo, mantienen retenidos a todos sus ancestros a través de sus respectivos wait(NULL) en cadena hasta llegar al proceso raíz.

1.4. Diagrama de secuencia



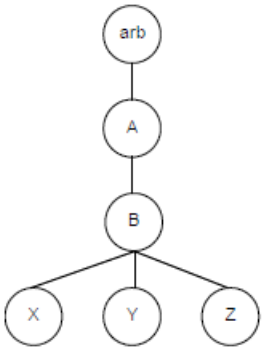
Ejercicio 1 (Apartado b)

1.5. Enunciado del Ejercicio

Realiza un programa llamado ejec.c que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando "pstree". El proceso Z no puede utilizar el comando "sleep", por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).

1.5.1. Estructura del Árbol de Procesos

La jerarquía de procesos a realizar es la siguiente:



1.6. Identificación y Justificación de Tareas

1.6.1. Recepción y Validación de Argumentos

El programa comprueba inicialmente que se haya pasado el tiempo por parámetro (argc < 2). Esto es necesario para configurar los segundos que debe esperar el temporizador.

1.6.2. Generación de la jerarquía de procesos anidada:

Para hacer la estructura de árbol que nos pide el enunciado, se realizan llamadas sucesivas a fork. Cada proceso hijo recién creado se encarga de registrar su identidad e invocar al siguiente nivel hasta completar el árbol.

1.6.3. Temporización

Ya que no podemos utilizar sleep(), programamos una alarma con alarm(segundos) y suspendiendo con pause() hasta que llegue la señal SIGALRM.

1.6.4. Ejecución del comando pstree

Cuando la alarma de Z termina, se activa su manejador, el cual utiliza la llamada kill() para enviar una señal de ejecución (SIGUSR1) al proceso A. Al recibirla, el manejador de A desdobla un proceso temporal mediante fork() que se transforma inmediatamente en el comando pstree utilizando execlp()

1.6.5. Destrucción controlada

Para evitar que los padres mueran antes que sus hijos, colocamos llamadas a wait(NULL) en cada proceso padre. De esta forma, me aseguro de que el árbol se vaya cerrando ordenadamente desde las hojas (los procesos de abajo) hacia la raíz.

1.7. Análisis Detallado del Código Fuente

A continuación, se desglosa el funcionamiento lógico del programa ejec.c y las llamadas al sistema utilizadas en cada bloque

1.7.1. Funciones manejadoras de señales

manejador_vacio()

Captura la señal SIGUSR2 para evitar que los procesos intermedios y hojas mueran por defecto al recibirla.

```c
void manejador_vacio(int sig) {

}
```

manejador_z()

Se activa cuando termina la alarma en el proceso Z y utiliza kill() para enviar la señal SIGUSR1 al proceso A

```c
void manejador_z(int sig) {
    kill(pid_A_global, SIGUSR1);
}
```

manejador_a()

Al recibir la señal de Z, desdobla un proceso temporal con fork() que ejecuta el comando pstree mediante execlp(), espera a que termine con wait() y avisa al proceso B utilizando kill()

```c
void manejador_a(int sig) {
    pid_t pid_hijo = fork();
    if (pid_hijo == 0) {
        execlp("pstree", "pstree", NULL);
        perror("Error al ejecutar pstree");
        exit(1);
    } else if (pid_hijo > 0) {
        wait(NULL);
        kill(pid_B_global, SIGUSR2);
    }
}
```

1.7.2. Función parse_args()

Se encarga de evaluar los argumentos pasados por la consola. Comprueba que se introduce el parámetro de los segundos necesarios; si no es así, muestra un mensaje de error y finaliza. Si es correcto, convierte el valor de texto a un entero utilizando atoi().

```c
void parse_args(int argc, char *argv[], int *segundos) {
    if (argc < 2) {
        printf("Uso: %s <segundos>\n", argv[0]);
        exit(1);
    }
    *segundos = atoi(argv[1]);
}
```

Llamadas utilizadas: **printf(), atoi(), exit()**

1.7.3. Funciones de los procesos hoja

ejecutar_hoja_x, ejecutar_hoja_y,

```c
void ejecutar_hoja_y(pid_t p_A, pid_t p_ejec) {
    printf("Soy el proceso Y: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", getpid(), getppid(), p_A, p_ejec);
    signal(SIGUSR2, manejador_vacio);
    pause();
    printf("Soy Y (%d) y muero\n", getpid());
    exit(0);
}


void ejecutar_hoja_x(pid_t p_A, pid_t p_ejec) {
    printf("Soy el proceso X: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", getpid(), getppid(), p_A, p_ejec);
    signal(SIGUSR2, manejador_vacio);
    pause();
    printf("Soy X (%d) y muero\n", getpid());
    exit(0);
}
```

Imprimen su información de identificación de PIDs, configuran su manejador vacío y se quedan suspendidas con pause() hasta recibir la orden de cierre.

ejecutar_hoja_z

```c
void ejecutar_hoja_z(pid_t p_A, pid_t p_ejec, int segundos) {
    printf("Soy el proceso Z: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", getpid(), getppid(), p_A, p_ejec);

    signal(SIGALRM, manejador_z);
    signal(SIGUSR2, manejador_vacio);

    alarm(segundos);
    pause();

    pause();
    printf("Soy Z (%d) y muero\n", getpid());
    exit(0);
}
```

Además de lo anterior, asocia la señal de alarma a su manejador mediante signal(), programa el temporizador con alarm(segundos) y utiliza un doble pause() para controlar los tiempos de espera antes de expirar y cerrar su ejecución.

1.7.4. Funciones de gestión del proceso B

crear_proceso_b

```c
void crear_proceso_b(pid_t p_A, pid_t p_ejec, int segundos) {
    pid_t pid_B = fork();
    if (pid_B < 0) {
        perror("Error en fork B");
        exit(1);
    }

    if (pid_B == 0) {
        crear_hojas_b(p_A, p_ejec, segundos);
    }

    pid_B_global = pid_B;
    wait(NULL);
}
```

crear_hojas _b

Realiza tres llamadas consecutivas a fork() para crear los procesos X, Y y Z. Tras esto, se queda bloqueado con pause(). Cuando recibe el aviso de cierre envía a sus hijos una señal de destrucción con kill() y recolectando sus estados de forma síncrona mediante waitpid() para evitar procesos "zombis" antes de terminar.

```c
void crear_hojas_b(pid_t p_A, pid_t p_ejec, int segundos) {
    pid_t p_B = getpid();
    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es %d\n", p_B, p_A, p_ejec);

    signal(SIGUSR2, manejador_vacio);

    // Creamos proceso X
    pid_t pid_X = fork();
    if (pid_X < 0) { perror("Error en fork X"); exit(1); }
    if (pid_X == 0) { ejecutar_hoja_x(p_A, p_ejec); }

    // Creamos proceso Y
    pid_t pid_Y = fork();
    if (pid_Y < 0) { perror("Error en fork Y"); exit(1); }
    if (pid_Y == 0) { ejecutar_hoja_y(p_A, p_ejec); }

    // Creamos proceso Z
    pid_t pid_Z = fork();
    if (pid_Z < 0) { perror("Error en fork Z"); exit(1); }
    if (pid_Z == 0) { ejecutar_hoja_z(p_A, p_ejec, segundos); }

    pause();

    kill(pid_Z, SIGUSR2);
    waitpid(pid_Z, NULL, 0);

    kill(pid_Y, SIGUSR2);
    waitpid(pid_Y, NULL, 0);

    kill(pid_X, SIGUSR2);
    waitpid(pid_X, NULL, 0);

    printf("Soy B (%d) y muero\n", getpid());
    exit(0);
}
```

1.7.5. Función crear_proceso_a()

Guarda el PID de A globalmente para q las hojas sepan donde mandar las señales, e inicializa el recibimiento de SIGUR1 con signal(). Luego llama a la función del proceso b y espera hasta que termine con un wait(NULL) antes de cerrar.

```c
void crear_proceso_a(pid_t p_ejec, int segundos) {
    pid_t p_A = getpid();
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n", p_A, p_ejec);

    pid_A_global = p_A;
    signal(SIGUSR1, manejador_a);

    crear_proceso_b(p_A, p_ejec, segundos);

    printf("Soy A (%d) y muero\n", getpid());
    exit(0);
}
```

1.7.6. Función main()

Comienza llamando a parse_args() para validar los datos de entrada, obtiene su propio PID y desdobla el primer nivel del árbol creando al proceso A con fork(). Finalmente, el proceso raíz original (ejec)

se queda retenido con un wait(NULL) y no hace su return 0 final hasta que el proceso A y todos sus hijos hayan terminado.

```c
int main(int argc, char *argv[]) {
    int segundos;
    parse_args(argc, argv, &segundos);

    pid_t pid_ejec = getpid();
    printf("Soy el proceso ejec: mi pid es %d\n", pid_ejec);

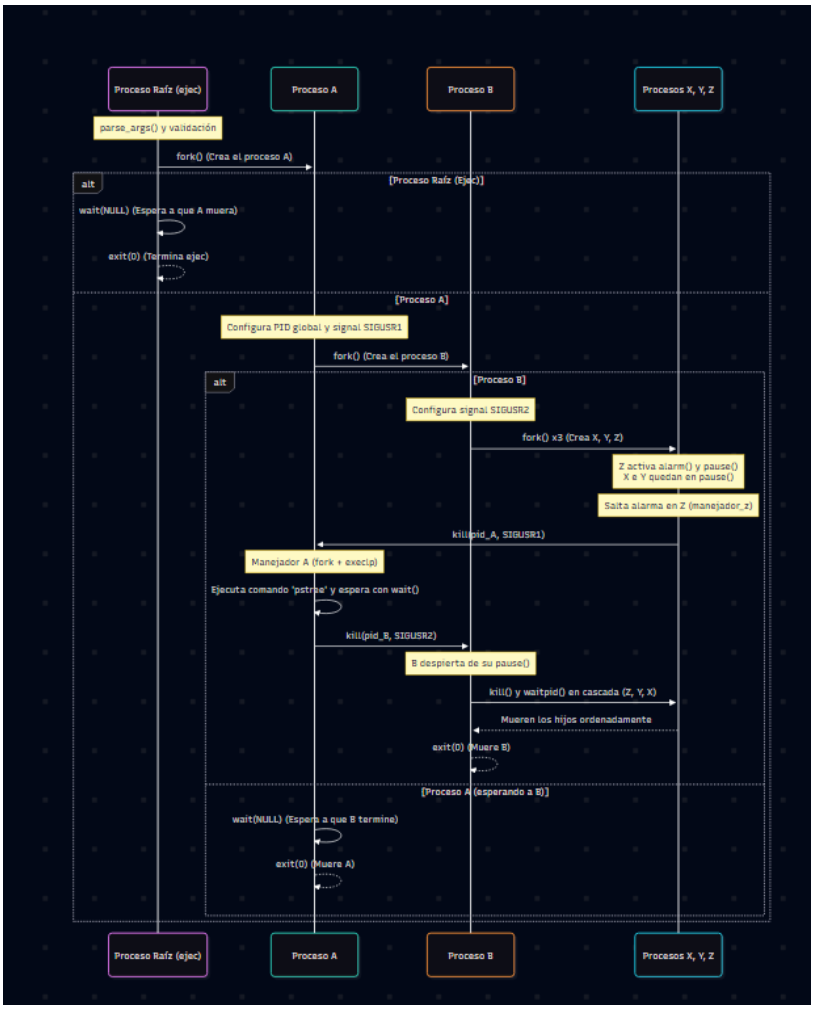
    // Creamos el proceso A
    pid_t pid_A = fork();
    if (pid_A < 0) {
        perror("Error en fork A");
        exit(1);
    }

    if (pid_A == 0) {
        crear_proceso_a(pid_ejec, segundos);
    }

    wait(NULL);
    printf("Soy ejec (%d) y muero\n", getpid());

    return 0;
}
```

1.8. Diagrama de secuencia



Ejercicio 2

2.1. Enunciado del Ejercicio

Realizar un programa llamado hacha.c que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, … . Se debe tener en cuenta el formato y consideraciones siguientes:

$ hacha <archivo> <tamaño>
<archivo> nombre del archivo a dividir
<tamaño> tamaño en bytes de los archivos divididos

Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

Consideraciones:

- Las entradas y salidas a los archivos se realizarán con llamadas al sistema estudiadas en la asignatura (no se puede utilizar printf, scanf, etc.)

- El padre creará tantos hijos como fragmentos del archivo a realizar, siendo decisión del alumno si los hijos se lanzan secuencial o concurrentemente.

2.2. Identificación y Justificación de Tareas

2.2.1. Recepción y Validación de Argumentos

La primera tarea es comprobar que el programa reciba exactamente el nombre del archivo y el tamaño de los fragmentos a través de la línea de comandos

2.2.2. Cálculo de fragmentos

El proceso original debe generar tantos hijos directos como columnas se hayan solicitado (y). Para ello, se utiliza un bucle donde el proceso padre ejecuta fork(). Es vital que los procesos hijos ejecuten un break para salir de este bucle y no crear columnas infinitas de forma descontrolada.

2.2.3. Mecanismo de división y comunicación (Padre e Hijos)

El padre es el único que interactúa directamente leyendo el archivo. Por cada fragmento, el padre crea una tubería (pipe) y genera un proceso hijo. El padre lee el trozo exacto de bytes que corresponde y lo envía por la tubería, mientras que el hijo recoge esos datos del canal y los guarda en su archivo destino correspondiente.

2.3. Análisis Detallado del Código Fuente

A continuación, se desglosa el funcionamiento lógico del programa malla.c y las llamadas al sistema utilizadas en cada bloque

2.3.1. Función parse_args()

Esta función revisa que el usuario ponga exactamente 3 argumentos (hacha, archivo, tamaño). Si los argumentos no son correctos utiliza write() para mandar el mensaje de error y se corta la ejecución con exit(1). Si los argumentos son correctos se guardan los valores usando los punteros que le hemos pasado desde el main.

```c
void parse_args(int argc, char *argv[], char **nombre_archivo, int *tam_trozo) {
    if (argc != 3) {
        char msg[] = "Uso: hacha <archivo> <tamaño>\n";
        write(1, msg, strlen(msg));
        exit(1);
    }
    // Asignamos los valores a las direcciones de memoria proporcionadas
    *nombre_archivo = argv[1];
    *tam_trozo = atoi(argv[2]);
}
```

- Llamadas al Sistema Utilizadas: **write(), exit()**

2.3.2. Función inicializar_datos_archivo()

Esta función utiliza la estructura stat para leer los datos del archivo y sacar su tamaño total en bytes con st_size. Para obtener el numero de hijos necesarios dividimos el tamaño total entre el tamaño del trozo. Si hay resto en la división sumamos un hijo.

```c
void inicializar_datos_archivo(char *nombre_archivo, int tam_trozo, int *tam_total, int *num_hijos) {
    struct stat info;
    if (stat(nombre_archivo, &info) < 0) {
        perror("Error al obtener info del archivo");
        exit(1);
    }

    *tam_total = info.st_size;
    *num_hijos = *tam_total / tam_trozo;

    if (*tam_total % tam_trozo != 0) {
        (*num_hijos)++;
    }
}
```

Llamadas al Sistema Utilizadas**: stat(), exit()**

2.3.3. Función abrir_archivo_origen()

Simplemente abre el archivo que queremos dividir en modo de solo lectura (O_RDONLY). Devuelve el descriptor del archivo para que el padre lo use más adelante.

```c
int abrir_archivo_origen(char *nombre_archivo) {
    int fd = open(nombre_archivo, O_RDONLY);
    if (fd < 0) {
        perror("Error al abrir origen");
        exit(1);
    }
    return fd;
}
```

Llamadas al Sistema Utilizadas**: open(), exit()**

2.3.4. Función crear_y_escribir_fragmento()

Es la función que realiza cada proceso hijo. Primero Cambia el nombre del archivo de destino y lo crea físicamente en el disco usando creat(). Luego entra en un bucle donde lee los datos que le llegan de la tubería (pipe_lectura) y los va escribiendo en su archivo nuevo con write(). Al terminar, cierra todo para liberar recursos.

```c
void crear_y_escribir_fragmento(char *nombre_archivo, int num_trozo, int pipe_lectura) {
    char nombre_destino[256];
    char buffer[1024];
    int fd_destino, leidos;

    sprintf(nombre_destino, "%s.h%02d", nombre_archivo, num_trozo);

    fd_destino = creat(nombre_destino, 0666);
    if (fd_destino < 0) {
        perror("Error al crear destino");
        exit(1);
    }

    while ((leidos = read(pipe_lectura, buffer, sizeof(buffer))) > 0) {
        write(fd_destino, buffer, leidos);
    }

    close(fd_destino);
    close(pipe_lectura);
}
```

Llamadas al Sistema Utilizadas**: creat(), read(), write(), close(), exit()**

2.3.5. Función leer_y_enviar_datos()

Esta es la función que realiza el padre. El padre lee la cantidad exacta de byte del archivo original usando read() y los va metiendo en el extremo de escritura de la tubería (pipe_escritura). Cuando termina de enviar su parte, cierra la tubería, lo que le avisa al hijo de que ya no hay más datos.

```c
void leer_y_enviar_datos(int fd_origen, int bytes_a_transferir, int pipe_escritura) {
    char buffer[1024];
    int leidos, por_leer;
    int total_enviados = 0;

    while (total_enviados < bytes_a_transferir) {
        por_leer = sizeof(buffer);
        if ((bytes_a_transferir - total_enviados) < sizeof(buffer)) {
            por_leer = bytes_a_transferir - total_enviados;
        }

        leidos = read(fd_origen, buffer, por_leer);
        if (leidos <= 0) break;

        write(pipe_escritura, buffer, leidos);
        total_enviados += leidos;
    }

    close(pipe_escritura);
}
```

Llamadas al Sistema Utilizadas**: read(), write(), close()**

2.3.6. Función generar_hijos_y_fragmentar()

Utilizando un bucle for, da tantas vueltas como hijos necesitemos. En cada vuelta:

- Crea una tubería nueva con pipe() antes de dividirse.
- Llama a fork() para desdoblar el proceso.
- El hijo cierra el lado de escritura de la tubería, llama a la función crear_y_escribir_fragmento y muere.
- El padre cierra el lado de lectura de la tubería, llama a la función leer_y_enviar_datos, y se queda bloqueado con wait(NULL) esperando a que este hijo en particular termine antes de dar la siguiente vuelta al bucle.

```c
void generar_hijos_y_fragmentar(char *nombre_archivo, int fd_origen, int tam_total, int tam_trozo, int num_hijos) {
    int tuberia[2];
    int bytes_para_hijo;

    for (int i = 0; i < num_hijos; i++) {
        if (pipe(tuberia) < 0) {
            perror("Error en pipe");
            exit(1);
        }

        if (i == num_hijos - 1 && tam_total % tam_trozo != 0) {
            bytes_para_hijo = tam_total % tam_trozo;
        } else {
            bytes_para_hijo = tam_trozo;
        }

        pid_t pid = fork();

        if (pid < 0) {
            perror("Error fork");
            exit(1);
        }

        if (pid == 0) {
            close(tuberia[1]);
            crear_y_escribir_fragmento(nombre_archivo, i, tuberia[0]);
            exit(0);
        } else {
            close(tuberia[0]);
            leer_y_enviar_datos(fd_origen, bytes_para_hijo, tuberia[1]);
            wait(NULL);
        }
    }
}
```

Llamadas al Sistema Utilizadas**: pipe(), fork(), close(), wait(), exit()**

2.3.7. Función main()

Declara las variables locales y va llamando a las funciones anteriores en orden pasándoles los parámetros por referencia. Al terminar de dividirlo todo cierra el archivo y finaliza.

```c
int main(int argc, char *argv[]) {

    char *nombre_archivo;
    int tam_trozo;
    int tam_total, num_hijos;

    parse_args(argc, argv, &nombre_archivo, &tam_trozo);

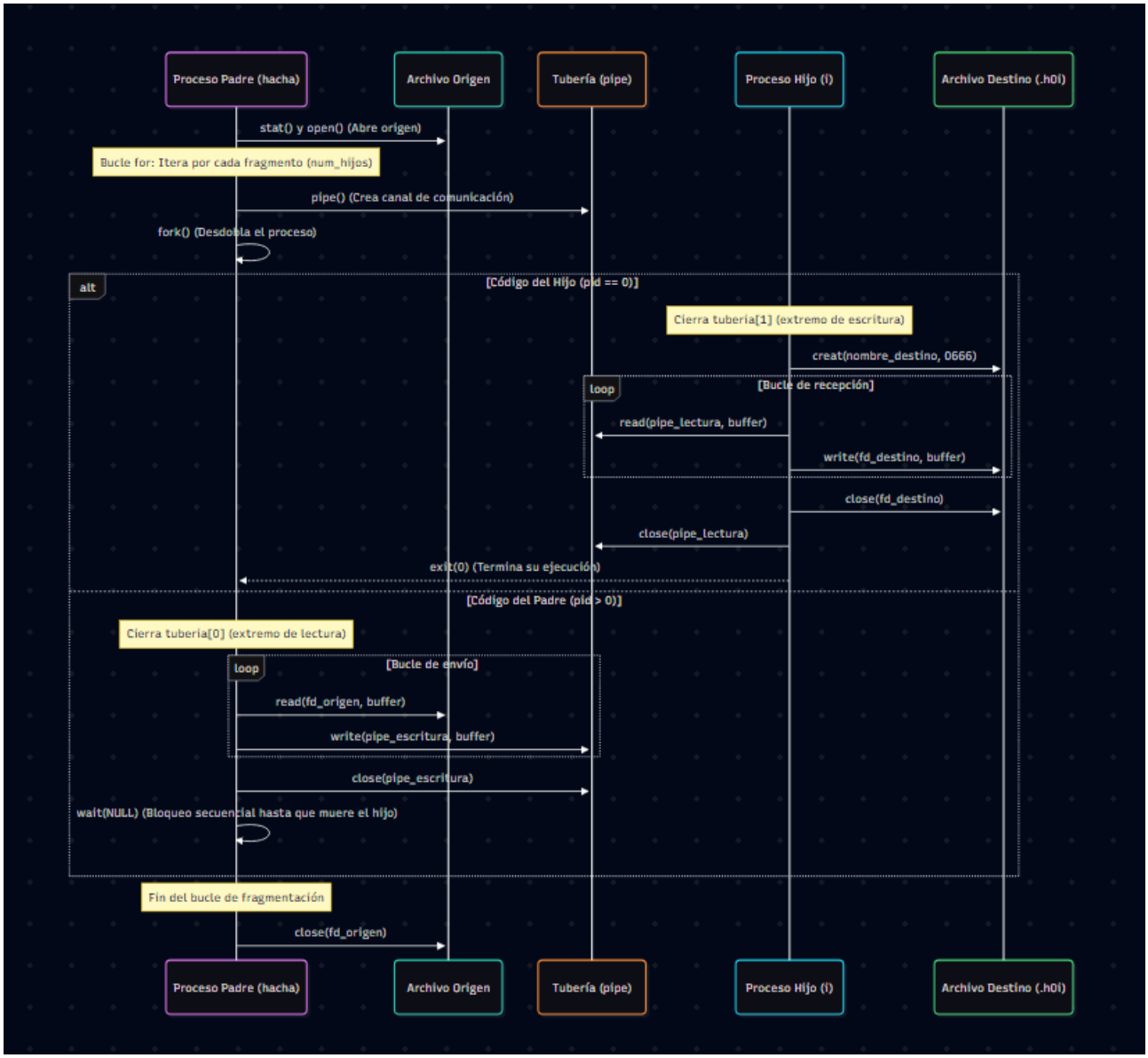
    inicializar_datos_archivo(nombre_archivo, tam_trozo, &tam_total, &num_hijos);
    int fd_origen = abrir_archivo_origen(nombre_archivo);

    generar_hijos_y_fragmentar(nombre_archivo, fd_origen, tam_total, tam_trozo, num_hijos);

    close(fd_origen);
    return 0;
}
```

Llamadas al Sistema Utilizadas**: close()**

2.4. Diagrama de secuencia



Ejercicio 3

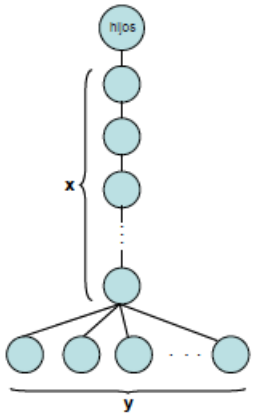
3.1. Enunciado del Ejercicio

Diseña un programa llamado hijos.c que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros

$hijos x y

3.4.1. Estructura del Árbol de Procesos

La jerarquía de procesos a realizar es la siguiente:



3.2. Identificación y Justificación de Tareas

3.2.1. Recepción y Validación de Argumentos

La primera tarea es comprobar que el programa reciba dos argumentos numéricos y que no sean números negativos o ceros.

3.2.2. Gestión de la Memoria Compartida

El proceso original debe generar tantos hijos directos como columnas se hayan solicitado (y). Para ello, se utiliza un bucle donde el proceso padre ejecuta fork(). Es vital que los procesos hijos ejecuten un break para salir de este bucle y no crear columnas infinitas de forma descontrolada.

3.2.3. Creación de la rama vertical (Padres):

Consiste en hacer un bucle donde cada proceso crea un hijo, guarda su PID y espera q su hijo termine, generando la cadena hacia abajo.

3.2.3. Creación de las hojas (Subhijos)

El último proceso de la cadena anterior se encarga de crear a los hijos finales, y luego los espera a todos.

3.3. Análisis Detallado del Código Fuente

A continuación, se desglosa el funcionamiento lógico del programa malla.c y las llamadas al sistema utilizadas en cada bloque

3.3.1. Funciones de configuración y memoria

validar_argumentos()

Revisa que argc sea 3 y convierte los textos a enteros. Si hay algún fallo, avisa usando write() y finaliza con exit().

```
void validar_argumentos(int argc, char *argv[], int *x, int *y) {
    if (argc != 3) {
        char error[] = "Uso: hijos <x> <y>\n";
        write(1, error, strlen(error));
        exit(1);
    }
    *x = atoi(argv[1]);
    *y = atoi(argv[2]);

    if (*x < 1 || *y < 1) {
        char error[] = "Error: x e y deben ser mayores que 0\n";
        write(1, error, strlen(error));
        exit(1);
    }
}
```

crear_memoria()

Calcula el tamaño total en bytes (x + y) * sizeof(int). Luego pide el segmento al sistema operativo usando shmget() y lo vincula al proceso con shmat() para que podamos usarlo como un puntero normal.

```c
int* crear_memoria(int x, int y, int *shmid) {

    int tamaño = (x + y) * sizeof(int);

    *shmid = shmget(IPC_PRIVATE, tamaño, IPC_CREAT | 0666);
    if (*shmid == -1) {
        perror("Error al crear memoria compartida");
        exit(1);
    }

    int *memoria = (int *) shmat(*shmid, 0, 0);
    if (memoria == (int *)-1) {
        perror("Error al vincular memoria");
        exit(1);
    }
    return memoria;
}
```

liberar_memoria()

Primero desvincula el puntero con shmdt() y después le dice al sistema que borre el segmento completamente usando shmctl() con la opción IPC_RMID

```c
void liberar_memoria(int shmid, int *memoria) {
    if (shmdt((char *) memoria) < 0) {
        perror("Error al desligar memoria");
        exit(1);
    }

    if (shmctl(shmid, IPC_RMID, 0) < 0) {
        perror("Error al borrar memoria");
        exit(1);
    }
}
```

3.3.2. Funciones auxiliares

registrar_padre() **y** ***registrar_hoja()***

Usan getpid() para sacar el identificador del proceso actual y lo guardan en su hueco correspondiente dentro de la memoria compartida

```c
void registrar_padre(int *memoria, int nivel) {
    memoria[nivel] = getpid();
}


void registrar_hoja(int *memoria, int x, int indice_hoja) {
    memoria[x + indice_hoja] = getpid();
}
```

imprimir_superpadre()* y *imprimir_subhijo()

Se encargan de recorrer el array de memoria compartida con un for y hacer los printf con el formato exacto que pide el enunciado

```c
void imprimir_superpadre(int *memoria, int x, int y) {
    printf("Soy el superpadre (%d): mis hijos finales son: ", getpid());
    for (int i = 0; i < y; i++) {
        printf("%d", memoria[x + i]);
        if (i < y - 1) printf(", ");
    }
    printf("\n");
}


void imprimir_subhijo(int *memoria, int x) {
    printf("Soy el subhijo %d, mi padres son: ", getpid());
    for (int i = 0; i < x; i++) {
        printf("%d", memoria[i]);
        if (i < x - 1) printf(", ");
    }
    printf("\n");
}
```

ejecutar_tarea_hoja()

Agrupa lo que hace un subhijo final: se registra, imprime su mensaje y muere con exit(0)

```c
void ejecutar_tarea_hoja(int *memoria, int x, int indice_hoja) {
    registrar_hoja(memoria, x, indice_hoja);
    imprimir_subhijo(memoria, x);
    exit(0); //
}
```

esperar_hijos_horizontales()

Es simplemente un bucle que hace wait(NULL) tantas veces como hijos horizontales haya para asegurarse de que todos terminan.

```c
void esperar_hijos_horizontales(int y) {
    for (int i = 0; i < y; i++) {
        wait(NULL); //
    }
}
```

3.3.3. Funciones de creación de procesos (Motor principal)

crear_hojas_horizontales()

Hace un bucle y veces llamando a fork(). Los procesos que nacen (hijos) van a su tarea, y el padre se queda esperando al final de la función.

```c
void crear_hojas_horizontales(int *memoria, int x, int y) {
    for (int i = 0; i < y; i++) {
        pid_t pid = fork(); //
        if (pid < 0) {
            perror("Error fork hoja");
            exit(1);
        }
        if (pid == 0) {
            ejecutar_tarea_hoja(memoria, x, i);
        }
    }
    esperar_hijos_horizontales(y);
}
```

crear_rama_vertical()

Hace un bucle x - 1 veces. Por cada vuelta hace un fork(). El comportamiento clave aquí es que el padre se queda bloqueado con wait(NULL) y el hijo es el que da la siguiente vuelta al bucle para seguir bajando niveles. El proceso que logra salir del bucle es el último de la fila, y es el que llama a crear las hojas

```c
void crear_rama_vertical(int *memoria, int x, int y) {
    for (int nivel = 0; nivel < x - 1; nivel++) {
        registrar_padre(memoria, nivel);

        pid_t pid = fork(); //
        if (pid < 0) {
            perror("Error fork vertical");
            exit(1);
        }

        if (pid > 0) {
            wait(NULL);

            if (nivel == 0) {
                imprimir_superpadre(memoria, x, y);
            }
            return;
        }
    }


    registrar_padre(memoria, x - 1);
    crear_hojas_horizontales(memoria, x, y);
}
```

main()

Solo declara las variables, llama a las funciones de validación y creación de memoria, lanza la función de crear la rama vertical, y al final comprueba si es el proceso original para borrar la memoria y salir.

```c
int main(int argc, char *argv[]) {
    int x, y, shmid;
    int *memoria;

    validar_argumentos(argc, argv, &x, &y);
    memoria = crear_memoria(x, y, &shmid);

    pid_t pid_original = getpid();


    crear_rama_vertical(memoria, x, y);

    if (getpid() == pid_original) {
        liberar_memoria(shmid, memoria);
    }

    return 0;
}
```

3.4. Diagrama de secuencia

